Quantum Computing and RTOS

Exploring real-time scheduling for quantum computing systems

Projektinformationen

Modul:	Echtzeitbetriebsysteme
Hochschule:	DHBW Ravensburg, Campus Friedrichshafen
Studiengang:	Embedded Systems
Dozent:	Dr. -Ing. Omar Alnaseri

Motorcycle

Dominik Borowski	7373896
Georg Martazov	7590033
Elias Sehmsdorf	2369656

Friedrichshafen, März 2026

1 Introduction

Quantum computing represents a fundamentally new paradigm of computation that leverages quantum mechanical phenomena such as superposition and entanglement to solve certain classes of problems more efficiently than classical computers. In recent years, significant advances in hardware development have enabled the construction of increasingly complex quantum processing units (QPUs). However, quantum computers do not operate in isolation. They rely heavily on classical control systems responsible for pulse generation, measurement processing, and error correction.

At the same time, real-time operating systems (RTOS) play a crucial role in classical computing environments where deterministic timing behavior and strict deadline guarantees are required. In embedded and safety-critical systems, real-time scheduling ensures that computational tasks are executed within well-defined temporal constraints. The concept of predictable and low-latency task execution is central to many modern control systems.

As quantum computing systems evolve, strict timing requirements emerge within the classical control layer. Qubits exhibit limited coherence times, and quantum gate operations must be executed with nanosecond precision. Furthermore, measurement results often need to be processed with minimal latency in order to enable feedback mechanisms and quantum error correction. These characteristics introduce real-time properties into quantum computing architectures and raise the question of how classical real-time scheduling principles can be applied or adapted in this context.

This report explores the relationship between quantum computing systems and real-time operating systems. It analyzes the real-time requirements that arise in quantum computing architectures, discusses the challenges of applying traditional scheduling approaches to quantum systems, and examines current and potential strategies for implementing real-time scheduling in the classical control infrastructure surrounding quantum processors.

2 Basics of Real-Time Operating Systems

The fundamental objective of a Real-Time Operating System (RTOS) is to execute computational tasks within strict temporal constraints. Unlike general-purpose operating systems, where the primary goals are to maximize throughput and ensure fairness among users, an RTOS prioritizes predictability, determinism, and bounded latency [1]. In real-time environments, the logical correctness of a computation is insufficient if the result is not delivered within a predefined time window, known as a deadline.

At the core of an RTOS lies its task scheduler, which allocates central processing unit (CPU) time according to specific scheduling algorithms, such as Rate Monotonic (RM) or Earliest Deadline First (EDF). The scheduler must ensure that critical tasks preempt lower-priority operations, strictly guaranteeing timely execution [2]. Real-time systems are traditionally classified into three categories based on the severity of the consequences when a deadline is missed:

- **Hard Real-Time:** Missing a deadline constitutes a total system failure and may lead to catastrophic consequences. Traditional examples include avionics flight control and automotive anti-lock braking systems.
- **Firm Real-Time:** A late result holds zero utility and is immediately discarded, leading to degraded system performance but not outright failure (e.g., dropping frames in a

multimedia stream).

- **Soft Real-Time:** The value of a computation decreases after the deadline has passed, but the result remains somewhat useful, typical in graphical user interfaces.

In the context of the highly sensitive control architectures required for modern quantum processors, strict hard real-time guarantees are absolutely essential to maintain system integrity and computational coherence.

3 Fundamentals of Quantum Computing

A classical computer processes information using bits that take on one of two distinct states, typically represented physically by voltage levels corresponding to 0 and 1. In contrast, a quantum computer operates using quantum bits, or qubits. A qubit can be physically realized in various systems, such as electron spin, trapped ions, photon polarization, or superconducting circuits.

Each qubit is described by two orthogonal basis states, commonly denoted as $|0\rangle$ and $|1\rangle$, which are comparable to the classical binary states. If a qubit is measured in one of these basis states, it cannot simultaneously be found in the other. However, unlike classical bits, qubits can exist in a superposition of both basis states.

Superposition can be illustrated by the analogy of a spinning coin. While a classical coin is either heads or tails when observed, a spinning coin appears to represent both states until it is observed. However, unlike a classical probabilistic mixture, quantum superposition represents a coherent linear combination of basis states that allows for interference effects.

Mathematically, the state of a single qubit is described as a vector in a two-dimensional Hilbert space:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \tag{1}$$

where α and β are complex probability amplitudes satisfying the normalization condition $|\alpha|^2 + |\beta|^2 = 1$.

By applying quantum gates, which are unitary operations on the state vector, it is possible to manipulate these superposition states. Computation in quantum systems therefore takes place through operations in vector space, enabling interference and entanglement. These properties form the fundamental basis of quantum computing and distinguish it from classical computation. [3]

4 Architecture of Quantum Computing System

The central component of a quantum computer is the Quantum Processing Unit (QPU). Qubits are physically realized and maintained within the QPU and are highly sensitive to environmental disturbances. To minimize thermal noise and suppress decoherence effects, QPUs are typically operated at cryogenic temperatures close to absolute zero.

In order to control the qubits, quantum gates are implemented using precisely shaped laser or microwave pulses, depending on the physical realization of the system. These pulses influence

properties such as the spin, polarization, or energy levels of the qubits. To generate pulses with specific frequency, amplitude, and phase characteristics, a sophisticated classical control system is required.

This control infrastructure consists of high-performance electronic components, including application-specific integrated circuits (ASICs) and field-programmable gate arrays (FPGAs), which are widely used for fast and deterministic signal processing. The generated control pulses enable the initialization, manipulation, and readout of qubits.

Typically, a qubit is initialized in its ground state $|0\rangle$, which is achieved by cooling the system to reduce thermal excitations. After initialization, quantum gates are applied via precisely timed control pulses to manipulate the quantum state. The state of a qubit is read out by measuring changes in the reflected or transmitted microwave signals, depending on the hardware architecture.

Due to the inherent instability of quantum states and the limited coherence time of qubits, the classical control system must process measurement results with minimal latency. In particular, quantum error correction and feedback operations require real-time signal processing to maintain computational accuracy. Therefore, deterministic timing and low-latency control are essential components of modern quantum computing architectures.[4]

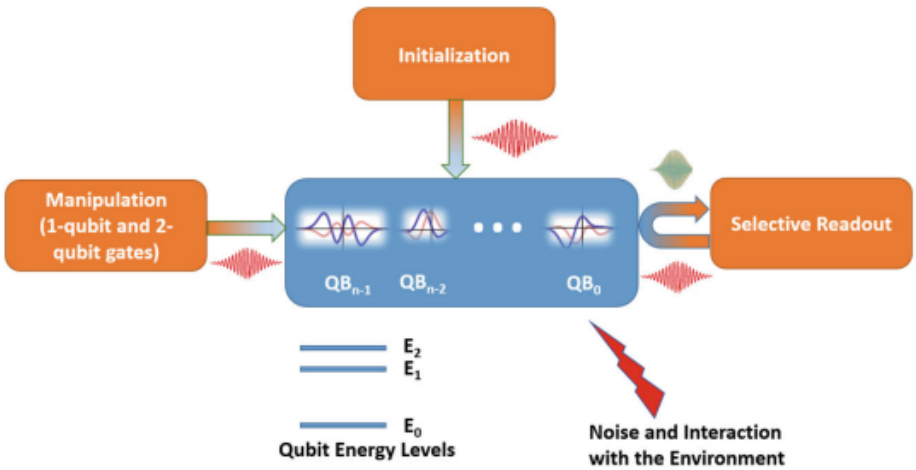

Figure 1: Operations on the Quantum Computing Architecture [4]

5 Real-Time Characteristics in Quantum Computing Systems

The operation of a quantum computer introduces unprecedented real-time constraints on its classical control infrastructure. Quantum bits (qubits), unlike classical transistors, are inherently fragile and extremely sensitive to environmental disturbances such as thermal noise, electromagnetic interference, and material defects. Interaction with the environment inevitably leads to decoherence, which corrupts the quantum superposition and entanglement. Depending on the physical implementation—such as superconducting circuits or trapped ions—the coherence time (T_1 relaxation and T_2 dephasing times) of a qubit frequently operates on the scale of microseconds to milliseconds. Consequently, all state preparation, logic gate execution, and measurement operations must be completed well within this narrow coherence window to avoid

catastrophic data loss. To execute quantum algorithms, the classical control system must apply precisely scheduled and shaped analog signals, typically in the form of microwave or laser pulses. These pulses induce the physical interactions required to perform single- and two-qubit gates. The timing of these pulses must be deterministic at the nanosecond scale; any timing jitter, clock skew, or latency in pulse generation introduces phase errors, diminishing gate fidelity and ultimately invalidating the computational accuracy. Furthermore, fault-tolerant quantum computing relies heavily on Quantum Error Correction (QEC) protocols, such as surface codes, which are vital for scaling quantum systems. QEC requires a continuous cycle of indirect syndrome measurements, rapid classical processing of the measurement outcomes, and the timely application of corrective feedback pulses.

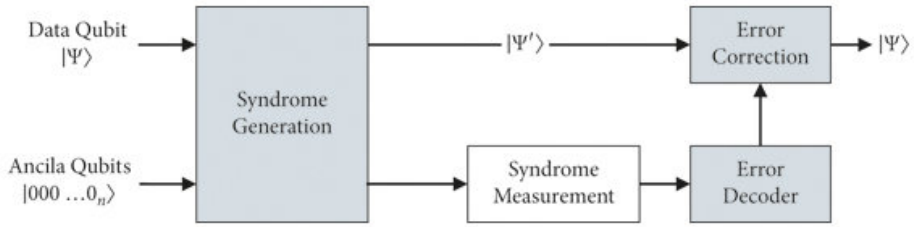

Figure 2: Digital System Design for Quantum Error Correction Codes [5]

6 Challenges in Real-Time Scheduling for Quantum Computing Systems

Real-time scheduling plays a crucial role in the classical control systems that operate quantum computers. Unlike conventional computing systems, quantum processors require extremely precise timing and coordination between hardware components, measurement systems, and classical control software. These strict requirements introduce several challenges for the design and implementation of real-time scheduling mechanisms.

Because the physical qubits continue to undergo rapid decoherence while the error syndrome is being measured and processed, the classical control system must decode the error and trigger the appropriate feedback loop in strict hard real-time [6]. This dynamic feedback loop commonly imposes extreme latency bounds in the range of hundreds of nanoseconds to a few microseconds. Failure to meet these stringent temporal deadlines renders the error correction entirely ineffective, leading to irrecoverable logical errors that propagate through the system. Thus, the classical infrastructure managing a Quantum Processing Unit (QPU) demands an execution environment where tasks, such as pulse scheduling and syndrome decoding, are guaranteed to complete within highly deterministic and exceptionally short timing windows. Conventional real-time operating systems must either be significantly adapted or tightly integrated with specialized hardware architectures, such as Field-Programmable Gate Arrays (FPGAs), to fulfill these rigorous real-time demands [7].

One of the most fundamental challenges arises from qubit decoherence. Qubits are highly sensitive to environmental disturbances and tend to lose their quantum state over time due to interactions with the surrounding environment. This process, known as decoherence, limits the time window in which quantum operations can be performed reliably [4]. As a result, control

operations, measurements, and feedback mechanisms must be executed within strict timing constraints to ensure that quantum information is not lost before the computation is completed.

Another challenge is the high sensitivity of quantum systems to noise and timing inaccuracies. Even small deviations in control signals or timing jitter can significantly affect the fidelity of quantum gates [6]. Since quantum algorithms rely on precise manipulation of qubit states, any timing variability in the classical control system may introduce errors in the computation. Consequently, scheduling mechanisms must ensure highly deterministic execution and minimal latency in the control pipeline.

A further complication arises from the need for real-time feedback and adaptive control. Many quantum computing architectures rely on measurement outcomes that must immediately influence subsequent operations [7]. This is particularly important in quantum error correction, where measurement results are used to detect and correct errors in qubit states. The classical controller must therefore process measurement data and trigger corrective actions within extremely short time intervals, often in the range of microseconds or less.

The increasing system complexity and scalability requirements also present major challenges. As quantum processors scale to larger numbers of qubits, the classical control infrastructure must coordinate an increasing number of parallel operations. These include pulse generation, qubit measurement, signal processing, and feedback control [4]. Scheduling these tasks while maintaining strict timing guarantees becomes significantly more difficult as system size grows.

In addition, the classical control systems of quantum computers typically consist of multiple hardware layers, including digital controllers, signal generators, measurement electronics, and cryogenic interfaces. The interaction between these layers introduces additional latency and synchronization challenges that must be considered when designing real-time scheduling mechanisms [7].

Overall, these challenges demonstrate that the classical control layer of a quantum computer must operate under strict real-time constraints. Ensuring deterministic task execution, minimizing latency, and maintaining precise synchronization between hardware components are essential requirements for reliable quantum computation [6].

7 Approaches to Real-Time Scheduling

To address the challenges described above, modern quantum computing platforms employ several approaches to achieve reliable real-time scheduling and control. These approaches combine specialized hardware, deterministic scheduling techniques, and tightly integrated control architectures.

One widely used approach is the use of low-latency control hardware, often implemented using Field Programmable Gate Arrays (FPGAs) [6]. FPGA-based controllers allow measurement signals to be processed and control pulses to be generated with extremely low latency. Because these devices operate at the hardware level, they can perform signal processing and feedback operations much faster than traditional software-based control systems. This capability is essential for implementing real-time feedback loops required for quantum error correction and adaptive quantum experiments [7].

Another important approach is the implementation of deterministic real-time scheduling algorithms. In real-time systems, tasks must meet strict deadlines to ensure correct system behav-

ior [2]. Scheduling strategies such as fixed-priority scheduling, deadline-based scheduling, or time-triggered scheduling can be used to guarantee predictable execution times. These methods ensure that critical control tasks are executed within their required time constraints [1].

In addition to deterministic scheduling, closed-loop feedback control plays a key role in many quantum computing systems [7]. In such systems, measurement results are continuously monitored and used to adjust subsequent operations. For example, when an error syndrome is detected in a quantum error correction scheme, corrective operations must be applied immediately to maintain the integrity of the quantum state [5]. This requires fast processing of measurement data and immediate scheduling of corrective tasks [6].

Another promising approach is hardware–software co-design. Instead of treating the classical control system and the quantum processor as separate components, modern architectures increasingly integrate them into a unified control stack [4]. In this approach, scheduling strategies, signal processing pipelines, and hardware interfaces are designed together to optimize system performance and minimize latency.

Finally, scalable quantum control architectures often incorporate hierarchical scheduling structures [7]. High-level software layers manage experiment configuration and algorithm execution, while lower-level real-time controllers handle time-critical tasks such as pulse generation and measurement processing. This hierarchical design allows complex quantum programs to be executed while maintaining strict timing guarantees at the hardware level [6].

By combining deterministic scheduling algorithms, specialized hardware controllers, and integrated system architectures, modern quantum computing platforms are able to meet the demanding real-time requirements of quantum control systems [1]. As quantum processors continue to scale, further research into efficient real-time scheduling and control strategies will remain essential for the development of large-scale quantum computing systems [4].

Practical Quantum Computing Resources

For those interested in exploring practical quantum computing implementations, IBM provides cloud-based access to real quantum processors through their IBM Quantum Platform [8]. This platform allows researchers and developers to experiment with quantum algorithms and observe the real-time control requirements discussed in this report on actual quantum hardware.

References

[1] H. Kopetz, *Real-Time Systems: Design Principles for Distributed Embedded Applications*. Springer Science & Business Media, 2nd ed., 2011.

[2] J. W. S. Liu, *Real-Time Systems*. Prentice Hall, 2000.

[3] H. Y. Wong, *Introduction to Quantum Computing*. Springer Nature, 2023.

[4] H. Y. Wong, *Quantum Computing Architecture and Hardware for Engineers*. Springer Nature, 2025.

[5] O. Khalifa, N. Sharif, R. Saeed, S. Abdel-Khalek, A. Alharbi, and A. Alkathiri, "Digital system design for quantum error correction codes," *Contrast Media & Molecular Imaging*, 12 2021.

[6] Y. Salathé *et al.*, "Low-latency digital signal processing for feedback in quantum computing and control," *Physical Review Applied*, vol. 9, no. 3, p. 034011, 2018.

[7] K. Reuer *et al.*, "Realizing repeated quantum error correction in a distance-three surface code," *Nature Communications*, vol. 14, p. 3148, 2023.

[8] IBM Quantum, "IBM Quantum Computing Platform." `https://quantum.cloud.ibm.com/`, 2026. Accessed: March 2026.